

Logistic Regression

1. Logistic Regression — What & Why

Linear regression predicts a number. But what if you want to predict YES or NO?

Logistic regression predicts the PROBABILITY of something belonging to a class. Output is always between 0 and 1.

3 types of classification:

- Binary classification — 2 classes only (spam / not spam, fraud / not fraud, sick / healthy)
- Multi-class classification — 3+ classes (cat, dog, rabbit — only one answer)
- Multi-label classification — multiple labels at once (a photo can be 'outdoor' AND 'sunny' AND 'beach')

Real example: Email spam detection

Input features: word count, sender domain, has link?, has attachment?

Output: probability = 0.87 → since 0.87 > 0.5 threshold → SPAM

probability = 0.12 → since 0.12 < 0.5 threshold → NOT SPAM

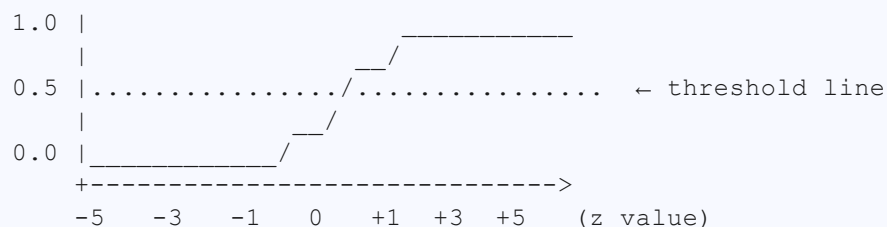
2. Sigmoid Function — The Core of Logistic Regression

Linear regression gives you $y = \beta_0 + \beta_1 x$. But that can give values like -5 or 200, which make no sense as a probability. So we pass it through the sigmoid function to squash it into [0, 1].

$$P(y=1) = 1 / (1 + e^{(-z)}) \quad \text{where } z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$

e = Euler's number ≈ 2.718 . No matter what z is, sigmoid always outputs between 0 and 1.

What the sigmoid curve looks like



$z = -5 \rightarrow P \approx 0.007$ (almost definitely class 0)

$z = 0 \rightarrow P = 0.5$ (50-50, exactly on decision boundary)

$z = +5 \rightarrow P \approx 0.993$ (almost definitely class 1)

Numerical example

Predicting if a student passes (1) or fails (0)

Fitted model: $z = -4 + 0.8 \times \text{hours_studied}$

Student A: studied 3 hours

$$z = -4 + 0.8 \times 3 = -4 + 2.4 = -1.6$$

$$P = 1 / (1 + e^{1.6}) = 1 / (1 + 4.95) = 0.168 \rightarrow \text{FAIL (below 0.5)}$$

Student B: studied 7 hours

$$z = -4 + 0.8 \times 7 = -4 + 5.6 = +1.6$$

$$P = 1 / (1 + e^{-1.6}) = 1 / (1 + 0.202) = 0.832 \rightarrow \text{PASS (above 0.5)}$$

3. Softmax — Multi-class Version of Sigmoid

When you have more than 2 classes, sigmoid won't work. You use softmax instead. It gives a probability for EACH class, and they all add up to 1.

$$P(\text{class } k) = e^{(z_k)} / \sum e^{(z_j)} \quad (\text{sum over all classes } j)$$

Example: Classifying a fruit (apple / banana / mango)

Raw scores (z): apple=2.0, banana=1.0, mango=0.5

$$e^{2.0} = 7.39, e^{1.0} = 2.72, e^{0.5} = 1.65$$

$$\text{Total} = 7.39 + 2.72 + 1.65 = 11.76$$

$$P(\text{apple}) = 7.39/11.76 = 0.628 \rightarrow 62.8\%$$

$$P(\text{banana}) = 2.72/11.76 = 0.231 \rightarrow 23.1\%$$

$$P(\text{mango}) = 1.65/11.76 = 0.140 \rightarrow 14.0\%$$

$$\text{Total} = 1.0 \quad \checkmark$$

Predicted class: APPLE (highest probability)

4. Decision Boundary

The decision boundary is the line (or curve) that separates class 0 from class 1. Everything on one side = class 1, the other side = class 0.

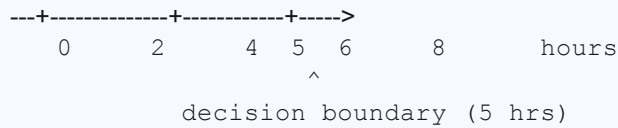
For sigmoid: the boundary is where $z = 0 \rightarrow P = 0.5$

$$\beta_0 + \beta_1 x = 0 \rightarrow x = -\beta_0 / \beta_1$$

Example: $z = -4 + 0.8 \times \text{hours}$

$$\text{Decision boundary: } -4 + 0.8x = 0 \rightarrow x = 5 \text{ hours}$$

$$\begin{array}{|c|c|c|} \hline \text{FAIL zone} & & \text{PASS zone} \\ \hline (P < 0.5) & & (P > 0.5) \\ \hline \end{array}$$



Study less than 5 hrs → predicted FAIL
Study more than 5 hrs → predicted PASS

5. Threshold — The Most Important Setting

Default threshold = 0.5. But you can change it. This is where things get really interesting.

The default rule: if $P \geq 0.5 \rightarrow$ predict class 1 (YES). If $P < 0.5 \rightarrow$ predict class 0 (NO).

But 0.5 is not always the right threshold! Changing it shifts the trade-off between catching more positives vs generating more false alarms.

Visual — what happens when you move the threshold

MODEL OUTPUT PROBABILITIES for 10 patients (disease screening):

Patient: A B C D E F G H I J
P(sick): 0.91 0.82 0.74 0.68 0.55 0.43 0.38 0.29 0.19 0.07
Actual: Sick Sick Sick Sick Well Well Sick Well Well Well

THRESHOLD = 0.5 (default):

Predict YES (sick) for: A B C D E (all ≥ 0.5)
Predict NO (well) for: F G H I J (all < 0.5)

Patient G is actually SICK but $P=0.38 < 0.5 \rightarrow$ predicted WELL ← FN!
Patient E is actually WELL but $P=0.55 > 0.5 \rightarrow$ predicted SICK ← FP!

THRESHOLD = 0.35 (lowered):

Predict YES for: A B C D E G (all ≥ 0.35)
Predict NO for: F H I J (all < 0.35)

Patient G now CAUGHT as sick → FN reduced ✓ (better recall)
But now F might get flagged → FP increases ✗ (worse precision)

THRESHOLD = 0.7 (raised):

Predict YES for: A B C (only ≥ 0.7)
Predict NO for: D E F G H I J

Patients D and G missed → FN increases ✗ (worse recall)
Almost no false alarms → FP very low ✓ (better precision)

The key rule to remember

LOWER threshold (e.g. 0.3):

- Model flags MORE things as positive
- FN decreases (catch more real cases – good for disease screening!)
- FP increases (more false alarms – annoying but less dangerous)

HIGHER threshold (e.g. 0.7):

- Model flags FEWER things as positive
- FP decreases (fewer false alarms – good for spam where wrong label is bad)
- FN increases (miss more real cases – dangerous in medical!)

RULE: You lower threshold when MISSING a real case is more dangerous than a false alarm.

You raise threshold when a false alarm is more costly than missing cases.

Medical diagnosis → lower threshold (don't miss sick patients)

Spam filter → higher threshold (don't mark real email as spam)

6. Cross-Entropy — Loss Function for Logistic Regression

MSE doesn't work well for classification. Logistic regression uses cross-entropy loss instead. It punishes confident wrong predictions very heavily.

$$\text{Loss} = -[y \times \log(\hat{y}) + (1-y) \times \log(1-\hat{y})]$$

y = actual label (0 or 1), $\hat{y}$ = predicted probability

Example: actual $y = 1$ (patient IS sick)

Model A predicts $\hat{y} = 0.9$ (confident and correct)

$$\text{Loss} = -[1 \times \log(0.9) + 0 \times \log(0.1)] = -\log(0.9) = 0.105 \leftarrow \text{small loss } \checkmark$$

Model B predicts $\hat{y} = 0.1$ (confident but WRONG)

$$\text{Loss} = -[1 \times \log(0.1) + 0 \times \log(0.9)] = -\log(0.1) = 2.303 \leftarrow \text{huge loss } \times$$

If you say a sick patient has only 10% chance of being sick → severe penalty.

This forces the model to be calibrated, not just directionally correct.

7. Confusion Matrix — Seeing Where the Model Goes Wrong

A confusion matrix shows all 4 types of outcomes: TP, TN, FP, FN. It's the foundation for all classification metrics.

	Predicted: YES (1)	Predicted: NO (0)
--	-----------------------	-------------------

Actual: YES (1)	TP <i>True Positive ✓ Correct fraud catch</i>	FN <i>False Negative ✗ Missed fraud (bad!)</i>
Actual: NO (0)	FP <i>False Positive ✗ Wrong alarm (costly)</i>	TN <i>True Negative ✓ Correct non-fraud</i>

What each cell means — fraud detection example

TP (True Positive) — Model said FRAUD, it IS fraud ← correct, great
 TN (True Negative) — Model said LEGIT, it IS legit ← correct, great
 FP (False Positive) — Model said FRAUD, but it's legit ← wrong, annoying customer
 FN (False Negative) — Model said LEGIT, but it's fraud ← WORST! money lost

Real numbers example (10,000 transactions):

TP = 400 (fraud caught)
 TN = 9400 (legit correctly cleared)
 FP = 100 (legit transactions blocked)
 FN = 100 (fraud that slipped through)

8. Accuracy, Precision, Recall, F1 — and When to Use Each

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Looks great but LIES when data is imbalanced. A model that says 'no fraud' every time gets 99.5% accuracy but catches zero fraud.

Using the numbers above: $(400 + 9400) / 10000 = 98.0\%$ ← looks amazing
 But we missed 100 frauds! Accuracy hides this completely.

Recall (Sensitivity) — 'Of all ACTUAL positives, how many did we CATCH?'

$$\text{Recall} = \frac{TP}{TP + FN}$$

Use when: missing a real positive is dangerous. Medical diagnosis, fraud detection, cancer screening.

Recall = $400 / (400 + 100) = 400/500 = 0.80 = 80\%$
 → We caught 80% of all actual frauds. 20% slipped through (FN).

To improve recall → LOWER the threshold
 More things get flagged as fraud → fewer FN → higher recall

Precision — 'Of all things we PREDICTED positive, how many were ACTUALLY positive?'

$$\text{Precision} = \frac{TP}{TP + FP}$$

Use when: a false alarm is costly. Spam filter — wrongly blocking real email is bad.

Precision = $400 / (400 + 100) = 400/500 = 0.80 = 80\%$
→ Of all transactions we flagged as fraud, 80% really were fraud.
20% were innocent customers we annoyed.

To improve precision → RAISE the threshold
Fewer things flagged → fewer FP → higher precision (but recall drops!)

F1 Score — When You Need Both

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Harmonic mean of precision and recall. Use when both matter equally and data is imbalanced.

$$F1 = 2 \times (0.80 \times 0.80) / (0.80 + 0.80) = 2 \times 0.64 / 1.60 = 0.80$$

If Precision = 1.0 but Recall = 0.1: $F1 = 2 \times (1.0 \times 0.1) / (1.0 + 0.1) = 0.18$ ← punishes imbalance
If both = 0.8: $F1 = 0.80$ ← rewards balance

Specificity — rarely asked but good to know

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP}) \quad \leftarrow \text{'of actual negatives, how many did we correctly label?'}$$

The Precision–Recall Tradeoff — summary

Threshold	TP	FP	FN	TN	Effect
0.3 (low)	High	High	Low	Low	Catches more fraud, more false alarms
0.5 (default)	Medium	Medium	Medium	Medium	Balanced — default starting point
0.7 (high)	Low	Low	High	High	Fewer false alarms, misses real fraud

The tradeoff: you CANNOT improve both precision and recall at the same time by just moving threshold.
Lowering threshold → recall goes UP, precision goes DOWN
Raising threshold → precision goes UP, recall goes DOWN

So you pick based on what your use case can afford to get wrong.

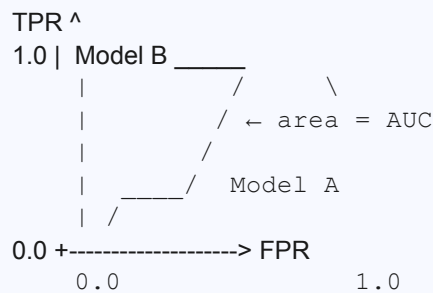
9. ROC Curve and AUC — Comparing All Thresholds at Once

Instead of picking one threshold and evaluating there, the ROC curve tests ALL possible thresholds from 0 to 1 and plots the result for each.

ROC curve plots: True Positive Rate (Recall) on y-axis vs False Positive Rate (1-Specificity) on x-axis

$$\begin{aligned} \text{TPR (y-axis)} &= \text{TP} / (\text{TP} + \text{FN}) && \leftarrow \text{same as Recall} \\ \text{FPR (x-axis)} &= \text{FP} / (\text{FP} + \text{TN}) && \leftarrow \text{false alarm rate} \end{aligned}$$

What the ROC curve looks like



Diagonal line = random guessing (AUC = 0.5) — useless model
Model A curve = decent model (AUC = 0.78)
Model B curve = great model (AUC = 0.95)
Perfect model = goes straight up then right (AUC = 1.0)

Each POINT on the curve = one threshold value
Bottom-left point = threshold 1.0 (predict nothing as positive)
Top-right point = threshold 0.0 (predict everything as positive)

AUC — Area Under the Curve

AUC = the area under the ROC curve. A single number that summarises model performance across all thresholds.

AUC = 0.5 → Random guessing. Model is useless.
AUC = 0.7 → Acceptable
AUC = 0.8 → Good
AUC = 0.9+ → Excellent
AUC = 1.0 → Perfect (probably overfitting!)

The beautiful thing about AUC: it doesn't depend on threshold choice.
You can compare two models without committing to a threshold.

How AUC works intuitively

AUC = probability that the model ranks a random positive higher than a random negative.

AUC = 0.95 means:

Take one actual fraud transaction and one actual legit transaction.
95% of the time, the model gives the fraud a higher probability score.
Only 5% of the time does it rank them wrongly.

Why AUC is better than accuracy for imbalanced data

Dataset: 9900 legit, 100 fraud

Dumb model: predict 'legit' for everything

Accuracy = 99% ← looks amazing

AUC = 0.5 ← tells the truth: it's useless

Good model: actually tries to find fraud

Accuracy = 97% ← looks worse!

AUC = 0.92 ← actually much better

Use AUC when classes are imbalanced. Accuracy alone will mislead you.

Quick Cheatsheet — Everything in One Place

Concept	Formula	Remember as
Sigmoid	$P = 1 / (1 + e^{(-z)})$	Squashes z into [0,1]
Softmax	$P(k) = e^{z_k} / \sum e^{z_j}$	Multi-class sigmoid, sums to 1
Cross-entropy loss	$-[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$	Punishes confident wrong predictions
Accuracy	$(TP+TN) / (TP+TN+FP+FN)$	Overall correct — misleading if imbalanced
Recall/Sensitivity	$TP / (TP+FN)$	Of actual positives, how many caught?
Precision	$TP / (TP+FP)$	Of predicted positives, how many real?
F1 Score	$2 \times P \times R / (P+R)$	Balance of precision and recall
Specificity	$TN / (TN+FP)$	Of actual negatives, how many correct?
Lower threshold	↓ threshold → ↑ Recall, ↓ Precision	Catch more, flag more
Higher threshold	↑ threshold → ↑ Precision, ↓ Recall	Less noise, miss more
AUC	Area under ROC curve (0.5-1.0)	Model quality across all thresholds